PYTHON SUMMER PLAN · DAY 05

Exceptions
& error handling

Catch what can fail. Raise meaningful errors. Build your own exception hierarchy.

Friday, May 15 · 75 minutes

Yuvraj Ruparel

R E C A P · Y E S T E R D A Y

Yesterday we raised. Today we catch.

Setters in shapes.py now raise ValueError and TypeError when given bad input. That's half the job. The other half: code that handles those errors gracefully — wraps them in context, recovers when possible, fails loud when not.

Y E S T E R D A Y

Raising

```
@radius.setter
def radius(self, value):
    if value <= 0:
        raise ValueError("radius > 0")
    self._radius = value
```

T O D A Y

Catching

```
try:
    c = Circle(pos, vel, radius=user_input)
except ValueError as e:
    print(f"bad radius: {e}")
    c = Circle.unit(pos)  # fallback
```

Raising tells callers something is wrong. Catching decides what to do about it.

try / except / else / finally

```
try:
    data = open(path).read()
    value = float(data)

except FileNotFoundError:
    value = 0.0          # specific first

except (ValueError, TypeError) as e:
    log(f"parse failed: {e}")
    value = 0.0

else:
    log(f"parsed {value}")  # only if no exception

finally:
    cleanup()                # always runs
```

The four blocks

- **try**

  The risky code. Keep it small — only the operation that might fail.

- **except**

  Run on a match. Specific types before broad ones. Multiple excepts allowed.

- **else**

  Runs only when try succeeded. Cleaner than putting success-path code inside try.

- **finally**

  Always runs — exception or not. For cleanup: close files, release locks.

EAFP vs LBYL

Two ways to handle things-that-might-fail. Python strongly prefers one.

LBYL (Look Before You Leap)

Check first, then act

```
if isinstance(value, (int, float)) and value > 0:
    c.radius = value
else:
    handle_bad_input()
```

Race conditions, duplicated checks (setter checks the same thing), and the test logic drifts from the actual logic over time.

EAFP (Easier to Ask Forgiveness)

Try it, handle failure

```
try:
    c.radius = value
except (ValueError, TypeError):
    handle_bad_input()
```

The setter is the single source of truth. Validation lives in one place. Callers state intent, not preconditions.

Default to EAFP in Python. Use LBYL only when the check is genuinely cheaper than attempting and failing.

raise, re-raise, and chain context

```
# 1. plain raise — bubble up unchanged
try:
    parse(line)
except ValueError:
    log("bad line")
    raise                          # re-raise current exception

# 2. raise from — chain context
try:
    radius = float(user_input)
except ValueError as e:
    raise InvalidDimensionError("radius must be numeric") from e

# 3. raise from None — hide the cause
    raise InvalidDimensionError(...) from None
```

Three patterns

- **Bare raise.**

  Re-raises whatever you just caught. Useful for logging then letting it propagate. Don't lose the traceback.

- **raise X from e.**

  New exception, chained. The traceback shows both — original cause plus your domain-specific wrapper. This is the default for translating errors.

- **raise X from None.**

  Suppresses the cause. Use rarely — when the original would leak implementation details (e.g. a database driver error becoming a clean DomainError).

Build a domain exception hierarchy

Generic ValueError is fine. But ShapeError tells callers exactly which subsystem failed — and they can catch broadly or narrowly.

```python
# exceptions.py
class ShapeError(Exception):
    """Base for all shapes.py errors."""

class InvalidDimensionError(ShapeError):
    """Negative or non-numeric size."""

class CollisionError(ShapeError):
    """Comparing incompatible shape types."""

# shapes.py
@radius.setter
def radius(self, value):
    if value <= 0:
        raise InvalidDimensionError(f"radius must be > 0, got {value}")
    self._radius = value
```

Why a hierarchy

Callers choose granularity:

except InvalidDimensionError catches one specific class of bug.

except ShapeError catches everything from your module — useful at API boundaries.

except Exception still works as a last resort.

Naming

Always end the class name with Error. Base class for your module ends with Error too — e.g. ShapeError, not Shape.

Fix vector3d.py normalize()

Yesterday's vector3d.py has a silent bug: normalize() of the zero vector returns NaN. Fix it the right way.

BEFORE

Silent failure

```
def normalize(self):
    m = self.magnitude
    return Vector3D(self.x/m, self.y/m, self.z/m)

>>> Vector3D(0,0,0).normalize()
Vector3D(nan, nan, nan)
```

Divides by zero, gets NaN, returns it as if nothing happened. The bug propagates downstream silently.

AFTER

Loud failure

```
def normalize(self):
    m = self.magnitude
    if m == 0:
        raise ZeroDivisionError("cannot normalize a zero vector")
    return Vector3D(self.x/m, self.y/m, self.z/m)
```

Fail at the source. Caller can catch it, recover, or let it crash loudly. NaN never enters the system.

Built-in exceptions worth knowing

EXCEPTION	WHEN IT FIRES	WHEN TO RAISE IT YOURSELF
ValueError	*Right type, wrong value.*	*Negative radius, empty list where one is needed.*
TypeError	*Wrong type entirely.*	*Setter got a string when it expected a number.*
ZeroDivisionError	*Division by zero.*	*normalize() of zero vector, dt = 0 in physics.*
AttributeError	*obj.x where x doesn't exist.*	*@property without @setter → read-only writes.*

Takeaway

Pick the most specific built-in that fits. If none fits, build a custom subclass of Exception in your module.

TODAY'S EXERCISE · 75 MIN

Harden shapes.py and vector3d.py

Build a real exception module. Replace generic ValueErrors with domain types. Fix the normalize() bug. Wrap the sandbox loop in a clean try/except.

SPEC

exceptions.py

ShapeError(Exception)

InvalidDimensionError(ShapeError)

CollisionError(ShapeError)

VectorError(Exception) for vector3d.py.

Wire up setters

Circle, Box, Shape setters raise InvalidDimensionError, not ValueError.

vector3d.normalize() raises VectorError on zero magnitude.

Chain context with raise X from e.

Sandbox handling

Wrap the main loop in try / except ShapeError.

Log the error, skip that frame, continue.

Use finally to plt.close() the figure.

Tips & gotchas

Never except: bare or except Exception broadly

You'll swallow KeyboardInterrupt, SystemExit, and bugs you didn't mean to catch. Always name the specific type. If you must catch broadly, log and re-raise.

Don't use exceptions for control flow

Exceptions are for exceptional cases — bad input, broken invariants. "Item not in dict" is normal; use dict.get() or in. Reserve raise for actual failures.

Keep the try block tiny

Only the operation that can fail. Everything else goes in else or after. The smaller the try, the easier it is to reason about which line raised.

Always include context in the message

raise ValueError(f"radius must be > 0, got {value!r}") tells you exactly what was wrong. "bad radius" tells you nothing at 2am.

W R A P - U P · W H A T T H I S F E E D S

Week 1 toolkit is complete

Classes, inheritance, composition, decorators, exceptions. Every piece you need to model a physical system in code. Tomorrow you use them all at once.

S A T U R D A Y

Spring-Mass-Damper

Full SMD class. Validated m, k, c. ζ = c / (2√(mk)) as @property. analytical_solution() vs numerical_solution(). Plot both. This week's capstone.

S U N D A Y

Buffer day

Catch-up or polish. Refactor anything that feels off. Re-read the OOP / decorator / exceptions notes.

W E E K 2

NumPy + plotting

ndarray instead of for-loops. Plot histories properly. Vectorize SMD for parameter sweeps. Week 1's classes don't disappear — they get faster.

When exceptions.py is clean and the sandbox recovers gracefully, drop the files back here.